STYLEHUB MEN FASHION E COMMERCE SYSTEM

PROJECT REPORT

Submitted by

Name: _____ Register Number: _____

Course Code and Title: _____

Guide: _____

Department: _____

College: _____

Month and Year: _____

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "StyleHub Men Fashion E Commerce System" is the bonafide work of _____ (Register Number _____) carried out under my supervision.

Guide Signature: _____

Head of Department Signature: _____

Date: _____

TABLE OF CONTENTS

Chapter	Title
1	Introduction
2	Objectives
3	Description
4	Class Diagram
5	Table Structures
6	ER Diagram
7	Tech Stack and Architecture
8	Modules
9	Conclusion

Page numbers can be updated after the final screenshots and cover details are inserted.

INTRODUCTION

Online fashion shopping gives customers access to many products, but choosing a suitable garment from pictures and product descriptions can still be difficult. A shopper must compare price, size, appearance, availability, and matching pieces while moving between multiple pages. A conventional catalogue provides the product record but does little to connect the selection, purchase, and personal styling decisions. StyleHub addresses this need through one men fashion web application that brings those activities together.

The system allows a registered user to browse a catalogue, inspect product details, view recently accessed items, save favourites, manage a cart, and purchase individual products, a coordinated outfit, or cart contents. It also adds social features: users can find friends, share selected wishlist products privately, and exchange reactions and comments. A rule based recommendation component suggests complementary garments for eligible products, while a deterministic chatbot answers common product and shopping questions. These two components use defined application logic rather than a trained conversational model.

For supported garments, StyleHub can send a customer photograph and a product garment image to a separate local FASHN VTON service and display the resulting virtual try-on image. This feature depends on the Python service, its model weights, and available hardware. The core shopping application continues to provide catalogue and purchase functions even when try-on is unavailable. Settings cover language, appearance, and notification preferences. An administrator has a restricted dashboard for product updates and operational summaries.

The application is implemented with a React and Vite frontend, a Node.js and Express REST API, and an SQLite database. JWT authentication identifies the user on protected requests, while the database records products, carts, purchases, social relationships, notifications, and try-on results. Optional provider integrations support Google sign-in and purchase communication when configured. This report documents the implemented modules, their data structures and APIs, and the interfaces to be illustrated by screenshots during final submission.

OBJECTIVES

- To provide secure registration, login, profile management, and protected user and administrator pages.
- To offer a men fashion catalogue with product details, stock, prices, and recently accessed items.
- To support size aware cart management, applicable rewards, checkout, and purchase history.
- To let users save favourites and revisit products without repeating a catalogue search.
- To allow users to connect with friends and privately share selected wishlist products.
- To record reactions and comments on shared selections for collaborative style decisions.
- To generate rule based complementary outfit suggestions for eligible products.
- To answer common product and shopping questions through a deterministic assistant.
- To offer virtual try-on for supported garments through a local Python service when available.
- To maintain notification records and user communication preferences.
- To give administrators a restricted dashboard for metrics, users, purchases, and product updates.
- To support English, Tamil, and Hindi interface options with a saved language choice.
- To keep user purchases and social records separated by authenticated account.

DESCRIPTION

StyleHub is a full stack men fashion application consisting of the following major functional areas. Each area is connected through the same user account and product catalogue.

1 User Authentication and Access Control

Registration and sign-in establish an account identity. Passwords are checked against stored hashes, and a JWT protects customer API requests. Profile editing and username completion support account maintenance, including Google-created accounts when that integration is configured. An ADMIN role controls access to the management dashboard.

2 Product Discovery and Personal Shopping

The catalogue displays product names, descriptions, images, categories, prices, stock, and applicable virtual try-on support. Product detail pages expose purchase, cart, wishlist, and outfit suggestion actions. Recently accessed records help a signed-in shopper return to previously viewed products. The cart retains selected products, sizes, and quantities before checkout.

3 Purchase and Communication

The purchase functions handle a single product, a coordinated combination, or all eligible cart items. They create purchase and purchase item records, calculate totals and applicable cart rewards, and update stock. A purchase history page lets the customer review saved transactions. In-app notifications and channel delivery records support purchase communication; external email, SMS, or WhatsApp delivery depends on configured providers.

4 Social Wishlist and Style Assistance

Wishlists store favourites separately from a cart. A user can send a friend request, accept a connection, share selected products, and respond to a shared list with feedback. Rule based outfit suggestions pair eligible products with complementary items. The chatbot answers defined styling and catalogue questions using application rules and product data.

5 Virtual Try-On and Administration

Virtual try-on accepts an uploaded customer image for a supported garment and requests local FASHN VTON inference. The generated result is saved for later viewing when the service succeeds. The administrator dashboard provides restricted access to metrics, catalogue updates, user summaries, purchase records, and try-on activity.

6 Technology and Preferences

React and Vite render the browser interface, Express exposes REST endpoints, and SQLite persists the application data. The interface supports English, Tamil, and Hindi and keeps the selected language. Appearance and communication settings are stored through the user settings flow. The optional Google sign-in and messaging integrations require separate provider credentials.

CLASS DIAGRAM

The accompanying editable Draw.io class diagram shows the main domain objects and their associations. User connects to cart, wishlist, purchase, friendship, and try-on records. Products are referenced by cart items, wishlist items, purchase items, and try-on results. The administrator is represented by the ADMIN role on a user account.

Fig 4.1 StyleHub Class Diagram

Open StyleHub_Class_Diagram.drawio to view and edit the UML structure. The diagram uses only black lines and text.

TABLE STRUCTURES

The following entries are taken from the SQLite schema in the project. PK denotes primary key and FK denotes foreign key. The tables store both shopping records and optional feature records such as social sharing and virtual try-on.

Cart

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
created_at	DATETIME		No
updated_at	DATETIME		No

Cart Items

Field	Type	Key	Required
id	INTEGER	PK	No
cart_id	INTEGER	FK → cart	Yes
product_id	INTEGER	FK → products	Yes
size	TEXT		No
quantity	INTEGER		Yes
created_at	DATETIME		No
updated_at	DATETIME		No

Friendships

Field	Type	Key	Required
id	INTEGER	PK	No
requester_id	INTEGER	FK → users	Yes
receiver_id	INTEGER	FK → users	Yes
status	TEXT		Yes
created_at	DATETIME		No
updated_at	DATETIME		No

Notification Deliveries

Field	Type	Key	Required
id	INTEGER	PK	No

Field	Type	Key	Required
purchase_id	INTEGER	FK → purchases	Yes
user_id	INTEGER	FK → users	Yes
channel	TEXT		Yes
status	TEXT		Yes
recipient	TEXT		No
provider_message_id	TEXT		No
error_message	TEXT		No
event_type	TEXT		No
subject	TEXT		No
created_at	DATETIME		No

Notifications

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
type	TEXT		Yes
title	TEXT		Yes
message	TEXT		Yes
is_read	INTEGER		Yes
created_at	DATETIME		No
related_id	INTEGER		No

Order Items

Field	Type	Key	Required
id	INTEGER	PK	No
order_id	INTEGER	FK → orders	Yes
product_id	INTEGER	FK → products	Yes
product_name	TEXT		Yes
product_image	TEXT		No
quantity	INTEGER		Yes
unit_price	REAL		Yes
discount	REAL		Yes
final_price	REAL		Yes
size	TEXT		No

Orders

Field	Type	Key	Required

Field	Type	Key	Required
id	INTEGER	PK	No
order_number	TEXT		Yes
user_id	INTEGER	FK → users	Yes
status	TEXT		Yes
subtotal	REAL		Yes
discount_total	REAL		Yes
shipping_fee	REAL		Yes
other_charges	REAL		Yes
total_amount	REAL		Yes
delivery_address	TEXT		Yes
created_at	DATETIME		No
updated_at	DATETIME		No
delivery_date	DATETIME		No
cancelled_at	DATETIME		No

Outfit Recommendations

Field	Type	Key	Required
id	INTEGER	PK	No
base_product_id	INTEGER	FK → products	Yes
recommended_product_id	INTEGER	FK → products	Yes
slot_type	TEXT		Yes
reason	TEXT		No
created_at	DATETIME		No
combination_signature	TEXT		No

Products

Field	Type	Key	Required
id	INTEGER	PK	No
name	TEXT		Yes
description	TEXT		No
brand	TEXT		No
category	TEXT		Yes
cloth_type	TEXT		No
color	TEXT		No
price	REAL		Yes
original_price	REAL		Yes
discount_percent	REAL		No
stock	INTEGER		Yes

Field	Type	Key	Required
status	TEXT		Yes
image	TEXT		No
garment_image	TEXT		No
vton_supported	INTEGER		No
vton_garment_category	TEXT		No
created_at	DATETIME		No
updated_at	DATETIME		No

Purchase Items

Field	Type	Key	Required
id	INTEGER	PK	No
purchase_id	INTEGER	FK → purchases	Yes
product_id	INTEGER	FK → products	Yes
quantity	INTEGER		Yes
unit_price	REAL		Yes
created_at	DATETIME		No
size	TEXT		No

Purchases

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
purchase_type	TEXT		Yes
total_amount	REAL		Yes
delivery_name	TEXT		Yes
delivery_phone	TEXT		Yes
delivery_address	TEXT		Yes
delivery_city	TEXT		Yes
delivery_state	TEXT		Yes
delivery_pincode	TEXT		Yes
created_at	DATETIME		No
cart_reward_percent	REAL		No
cart_reward_amount	REAL		No
subtotal	REAL		No

Recently Accessed

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
product_id	INTEGER	FK → products	Yes
accessed_at	DATETIME		No

Try On Results

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
product_id	INTEGER	FK → products	Yes
user_image_path	TEXT		Yes
garment_image_path	TEXT		Yes
generated_image_path	TEXT		No
category	TEXT		Yes
status	TEXT		Yes
created_at	DATETIME		No

Users

Field	Type	Key	Required
id	INTEGER	PK	No
username	TEXT		No
full_name	TEXT		Yes
email	TEXT		Yes
password_hash	TEXT		Yes
role	TEXT		Yes
phone	TEXT		No
age	INTEGER		No
address	TEXT		No
city	TEXT		No
state	TEXT		No
pincode	TEXT		No
created_at	DATETIME		No
updated_at	DATETIME		No
google_id	TEXT		No
avatar_url	TEXT		No
email_notifications	INTEGER		Yes

Field	Type	Key	Required
sms_notifications	INTEGER		Yes
whatsapp_notifications	INTEGER		Yes
theme_preference	TEXT		Yes
language_preference	TEXT		Yes

Wishlist Feedback

Field	Type	Key	Required
id	INTEGER	PK	No
share_id	INTEGER	FK → wishlist_shares	Yes
user_id	INTEGER	FK → users	Yes
reaction	TEXT		No
comment	TEXT		No
created_at	DATETIME		No
updated_at	DATETIME		No

Wishlist Items

Field	Type	Key	Required
id	INTEGER	PK	No
wishlist_id	INTEGER	FK → wishlists	Yes
product_id	INTEGER	FK → products	Yes
created_at	DATETIME		No

Wishlist Share Items

Field	Type	Key	Required
id	INTEGER	PK	No
share_id	INTEGER	FK → wishlist_shares	Yes
product_id	INTEGER	FK → products	Yes
created_at	DATETIME		No

Wishlist Shares

Field	Type	Key	Required
id	INTEGER	PK	No
sender_id	INTEGER	FK → users	Yes
receiver_id	INTEGER	FK → users	Yes
wishlist_id	INTEGER	FK → wishlists	No
share_type	TEXT		No
created_at	DATETIME		No

Wishlists

Field	Type	Key	Required
id	INTEGER	PK	No
user_id	INTEGER	FK → users	Yes
created_at	DATETIME		No
updated_at	DATETIME		No

CHAPTER 6

ER DIAGRAM

The accompanying editable Draw.io ER diagram summarizes the main foreign key relationships. It includes shopping, social, recommendation, notification, recently accessed, and try-on data. One user can have many purchases; each purchase has many purchase items. Product references link items and feature records back to the catalogue.

Fig 6.1 StyleHub Entity Relationship Diagram

Open StyleHub_ER_Diagram.drawio to inspect and edit the black-and-white entity relationship diagram.

CHAPTER 7

TECH STACK AND ARCHITECTURE

Technology Stack

Layer	Technology	Purpose
Frontend	React 18, Vite 6, React Router	Screens, navigation, user state
Backend	Node.js, Express 4	REST API and business rules
Database	SQLite, better-sqlite3	Persistent records and transactions
Authentication	JWT, bcryptjs	Tokens and password hashing
Virtual try-on	Python Flask, FASHN VTON 1.5	Local image inference for supported garments
Communication	Nodemailer, Twilio SMS, Meta WhatsApp API	Optional configured purchase notifications

Architecture Overview

The React browser application communicates with Express through REST endpoints. Express applies authentication and role checks, reads and writes SQLite records, and invokes business services. It calls the local VTON process for supported garment try-on. Configured external services support Google sign-in and notifications.

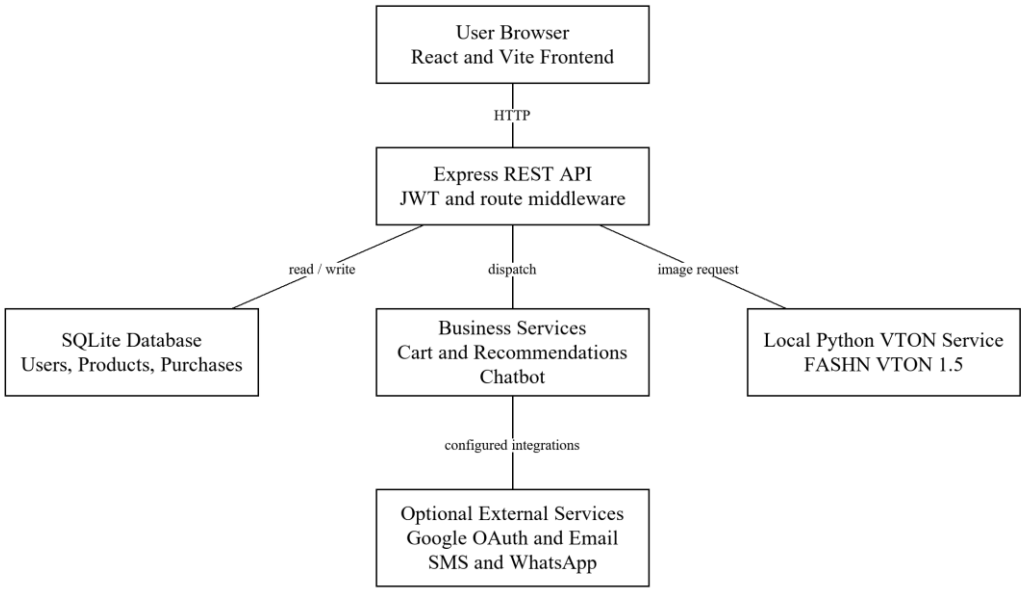


Fig 7.1 StyleHub System Architecture

The diagram shows request flow between the browser, Express API, SQLite database, business services, local VTON process, and optional external integrations. The editable source is StyleHub_Architecture.drawio.

CHAPTER 8

MODULES

8.0 Design Approach

The modules are described in a consistent order so that their responsibilities and implementation can be compared. Each section starts with its purpose, then lists observable functions, quality requirements, technologies, request flow, API endpoints, database design, and interfaces. The descriptions are based on the project routes, screens, and SQLite schema.

Design concern	Approach used in StyleHub
Identity	JWT identifies the user on protected requests; ADMIN routes require an administrator role.
Data integrity	SQLite foreign keys connect user, product, cart, purchase, social, and try-on records.
Modularity	React pages call frontend services; Express routes delegate application operations.
User isolation	The backend uses the authenticated user ID for account-specific records.
Optional services	VTON and provider communication report availability or delivery outcome separately.

Overall Request Flow

A user action begins in a React page. The frontend sends an HTTP request through its API service. Express selects a route, checks authentication where required, then performs validation and database or service work. The API returns a status and data to the page, which updates the displayed state. The local VTON service and external providers are called only for features that need them.

Step	Component	Result
1	React page or component	Collects input and starts an action
2	Frontend API service	Sends request and JWT when protected
3	Express route and middleware	Selects handler and checks access
4	Controller and service	Validates and applies business rules
5	SQLite or optional service	Reads, writes, or generates a result
6	API response and React state	Shows data, confirmation, or an error

Fig 8.0.1 Overall Request Flow of StyleHub

The table follows an action from the browser to the API.
Middleware verifies protected access before a handler runs.
The handler uses SQLite or an optional service when needed.
The response updates the user interface or reports a failure.

8.1 Folder Structure

stylehub/stylehub-frontend contains React pages, reusable components, contexts, and frontend service calls. stylehub/stylehub-backend contains Express routes, controllers, services, middleware, database configuration, and tests. The VTON service is housed under the backend vton-service directory.

Fig 8.1.1 Overall Project Folder

The top-level project separates the application and report files.
The frontend and backend sit under the stylehub directory.
Each side has its own package and source organization.
This view helps locate the major project areas.

Fig 8.1.2 Backend Folder

The backend contains server.js and its application packages.
Routes receive requests and controllers perform operations.
Services hold shared rules and integrations.
The config directory contains SQLite initialization.

Fig 8.1.3 Backend Config Folder

The config folder holds database and JWT support files.
Database initialization defines the tables and indexes.
The SQLite file persists local application records.
Configuration is kept separate from request handlers.

Fig 8.1.4 Backend Routes Folder

Route files group endpoints by feature.

Each route maps an HTTP method and path to a handler.

Protected routes apply authentication middleware.

The groups include users, shopping, social functions and administration.

Fig 8.1.5 Backend Controllers Folder

Controllers validate inputs and assemble responses.

Each controller belongs to a feature area.

They use database and service functions as needed.

This separation keeps route definitions concise.

Fig 8.1.6 Backend Services Folder

Services contain reusable business and integration logic.

Recommendation rules and communication handling are examples.

They support the controllers without defining page layout.

Provider behaviour depends on configuration.

Fig 8.1.7 Frontend Folder

The frontend package contains the Vite application.

Public assets support product and interface imagery.

The src directory contains executable React code.
Its package file identifies React and build dependencies.

Fig 8.1.8 Frontend Source Folder

The src directory separates pages, components and contexts.
Services centralize browser requests to the API.
The application route map is defined in App.jsx.
This layout makes feature screens easier to find.

Fig 8.1.9 Frontend Pages Folder

Pages represent the main browser routes.
Shopping, social, profile, admin and try-on screens appear here.
ProtectedRoute guards signed-in and ADMIN pages.
A page can reuse components and contexts.

Fig 8.1.10 Frontend Components Folder

Components hold reusable interface elements.
Examples include the layout, product card and chatbot.
Shared display logic stays out of page files.
Components receive data from props and contexts.

Fig 8.1.11 Frontend Context Folder

Contexts hold state shared across screens.
Authentication, cart, wishlist and friends use contexts.
Language and appearance preferences are also managed here.
This avoids passing state through many page levels.

Fig 8.1.12 Frontend Services Folder

Service modules call the backend API.
They group cart, wishlist, friends, notification and try-on requests.
Pages use these functions to perform actions.
Response data is then reflected in the interface.

8.2 Authentication and Profile

8.2.0 Purpose and Scope

This module creates and maintains customer identities, protects account data and distinguishes an administrator from a customer. It covers registration, credential login, profile updates, username availability and the optional Google sign-in path.

8.2.1 Functional Requirements

- Register a user with required identity and credential details.
- Authenticate a returning user and return a signed session token.
- Load and update only the signed-in user profile.
- Allow a Google-created account to complete a unique username.
- Restrict administrator pages and APIs to accounts with the ADMIN role.

8.2.2 Non Functional Requirements

- Store password hashes rather than plaintext passwords.

- Reject missing or invalid tokens on protected endpoints.
- Keep one user from editing another user profile.

8.2.3 Tech Stack

React, React Router, Express, JWT, bcryptjs and SQLite.

8.2.4 Low Level Architecture

- The registration or login page sends form data to an auth route.
- The controller validates input, checks the users table and creates or verifies credentials.
- A successful login returns a JWT; ProtectedRoute and API middleware use it on later requests.
- An ADMIN role is checked again on the backend before dashboard operations.

8.2.5 API Design

Method	Endpoint	Purpose
POST	/api/auth/register	Create customer account
POST	/api/auth/login	Authenticate credentials
GET	/api/auth/me	Return current user
PUT	/api/auth/profile	Update current profile
GET	/api/auth/check-username	Check username availability
PUT	/api/auth/username	Save username

8.2.6 DB Design

Table	Key Fields	Use
users	id; email; username; password_hash; role	Account and access attributes

8.2.7 Screenshots

Fig 8.2.1 Registration Page

Shows account fields and validation feedback.

Collects a new customer identity.

Submits the details to the registration API.

Successful registration enables sign-in.

Fig 8.2.2 Login Page

Shows the credential entry form.

Submits the account email and password.

Displays a response if authentication fails.

Successful login opens protected shopping pages.

Fig 8.2.3 Profile and Username Page

Displays the current account details.

Allows supported profile fields to be changed.

Checks a proposed username for availability.

Stores the completed username for the account.

Fig 8.2.4 Admin Login Page

Provides a separate administrator entry point.

Uses the existing account role for access.

Rejects a user without ADMIN permission.

Leads to the restricted dashboard after login.

8.3 Product Catalogue and Recent Views

8.3.0 Purpose and Scope

This module presents products for discovery and records which items a signed-in user has viewed. Product records include catalogue text, images, prices, available stock and virtual try-on eligibility.

8.3.1 Functional Requirements

- List available catalogue products for browsing.
- Open a detail page for a selected product.
- Show category, price, discount, stock and garment imagery.
- Record and retrieve recently accessed items per user.
- Expose the selected product to cart, wishlist, recommendation and try-on actions.

8.3.2 Non Functional Requirements

- Return a clear result for an invalid product identifier.
- Keep recently accessed records isolated by user.
- Show stock and availability from persisted product data.

8.3.3 Tech Stack

React pages and product components, Express product routes, SQLite.

8.3.4 Low Level Architecture

- The catalogue page requests the product list from the Express API.
- Selecting an item requests its detailed record.
- The recent-access route updates the signed-in user and product pair.
- The page renders available actions based on the returned product fields.

8.3.5 API Design

Method	Endpoint	Purpose
GET	/api/products	List products
GET	/api/products/:id	Get product detail
GET	/api/recently-accessed	List user recent views
POST	/api/recently-accessed	Record a product view

8.3.6 DB Design

Table	Key Fields	Use
products	id; name; category; price; stock; image; vton_supported	Catalogue source
recently_accessed	user_id; product_id; accessed_at	Per-user viewing history

8.3.7 Screenshots

Fig 8.3.1 Home and Featured Products

Introduces the fashion collection.

Shows selected catalogue entries.

Provides navigation to the full shop.

Shows recent items when available.

Fig 8.3.2 Product Catalogue

Presents product cards for browsing.

Shows image, name, price and stock state.

Allows opening an individual item.

Supports the discovery stage of shopping.

Fig 8.3.3 Product Detail

Shows details for a chosen product.

Presents garment attributes and actions.

Offers cart, wishlist or purchase choices.

May show recommendation or try-on availability.

Fig 8.3.4 Recently Accessed Products

Lists products viewed by this account.
Uses saved access timestamps for ordering.
Allows a shopper to reopen an item.
Keeps the history separate from other users.

8.4 Cart and Checkout

8.4.0 Purpose and Scope

The cart groups selected products and size variants before purchase. Checkout calculates an order summary and applies the implemented Buy More Save More reward when the cart meets its rule conditions.

8.4.1 Functional Requirements

- Add a product and available size to the active cart.
- Change quantity or remove an item.
- Show subtotal and applicable cart reward.
- Submit eligible cart contents with delivery details.
- Create purchase records and decrement stock through the purchase path.

8.4.2 Non Functional Requirements

- Do not allow nonpositive quantities or unavailable stock to be purchased.
- Keep each cart tied to its authenticated owner.
- Perform stock and purchase updates in the same transaction where implemented.

8.4.3 Tech Stack

React CartContext and checkout pages, Express cart and purchase routes, SQLite.

8.4.4 Low Level Architecture

- The product page adds an item to the authenticated user cart.
- Cart routes return current items and accept quantity or removal changes.
- Checkout presents delivery details and computed totals.
- The cart purchase route validates items, records the purchase and updates stock.

8.4.5 API Design

Method	Endpoint	Purpose
GET	/api/cart	Read current cart
POST	/api/cart/items	Add a cart item
PUT	/api/cart/items/:productId	Change quantity
DELETE	/api/cart/items/:productId	Remove item
POST	/api/purchases/cart	Purchase cart contents

8.4.6 DB Design

Table	Key Fields	Use
cart	id; user_id	One active user cart
cart_items	cart_id; product_id; size; quantity	Selected variants
purchases	user_id; total_amount; cart_reward_amount	Completed purchase
purchase_items	purchase_id; product_id; size; quantity	Purchased units

8.4.7 Screenshots

Fig 8.4.1 Add to Cart Action

Starts from a product detail view.

Collects a supported size if required.

Adds the selected product to the cart.

Updates the cart shown to the signed-in user.

Fig 8.4.2 Cart Items and Quantities

Lists products in the active cart.

Displays size and quantity for each line.

Supports quantity changes and removal.

Updates the total after a change.

Fig 8.4.3 Cart Reward Summary

Shows the current subtotal.
Displays the applicable reward calculation.
Makes the final amount visible before checkout.
Changes when eligible cart contents change.

Fig 8.4.4 Checkout Page

Collects the delivery details.
Shows the purchase summary.
Submits an authenticated cart purchase.
Returns a confirmation or validation message.

8.5 Purchases and Notifications

8.5.0 Purpose and Scope

This module stores completed purchases and their items, provides a purchase history and records in-app and configured channel communication. Single, combination and cart purchase routes share the saved purchase model.

8.5.1 Functional Requirements

- Purchase one product, an eligible combination or a cart.
- Save delivery data, purchased items and final amount.
- Retrieve a signed-in user purchase history and one purchase detail.
- Create and mark in-app notifications as read.
- Record email, SMS or WhatsApp delivery attempts and outcomes.

8.5.2 Non Functional Requirements

- Return only purchases owned by the authenticated user.

- Preserve the purchase when a nonessential external notification fails.
- Record configured, disabled or failed delivery states accurately.

8.5.3 Tech Stack

Express purchase and notification routes, SQLite, Nodemailer, Twilio SMS and Meta WhatsApp when configured.

8.5.4 Low Level Architecture

- A purchase route validates product selection, stock and delivery fields.
- The database stores a purchase and its line items, with associated stock changes.
- The notification service creates an in-app message and attempts enabled channels.
- History routes return records for the authenticated account.

8.5.5 API Design

Method	Endpoint	Purpose
POST	/api/purchases/product	Purchase one product
POST	/api/purchases/combo	Purchase an outfit
POST	/api/purchases/cart	Purchase cart
GET	/api/purchases	List own purchases
GET	/api/purchases/:id	Get one purchase
GET	/api/notifications	List notifications
PUT	/api/notifications/:id/read	Mark notification read

8.5.6 DB Design

Table	Key Fields	Use
purchases	id; user_id; purchase_type; total_amount	Purchase header
purchase_items	purchase_id; product_id; quantity; unit_price	Purchase lines
notifications	user_id; title; message; is_read	In-app messages
notification_deliveries	purchase_id; channel; status; provider_message_id	Channel delivery audit

8.5.7 Screenshots

Fig 8.5.1 Single Product Purchase

Starts from one selected product.

Collects delivery details for checkout.

Shows the price before submission.

Creates a purchase for that item.

Fig 8.5.2 Complete Look Purchase

Shows the coordinated garment selection.

Allows purchasing the eligible combination.

Calculates the combined amount.

Stores each selected product as a purchase item.

Fig 8.5.3 Purchase History

Lists saved purchases for the user.

Shows amount and purchase information.

Allows opening a selected record.

Keeps records tied to the signed-in account.

Fig 8.5.4 Notification Panel

Lists in-app purchase messages.

Shows read and unread state.

Allows a message to be marked read.

External channel delivery is audited separately.

8.6 Wishlist and Friends

8.6.0 Purpose and Scope

The social module stores favourite products, manages friendship requests and supports private sharing with feedback. It keeps a user wishlist separate from the cart and limits sharing to permitted relationships.

8.6.1 Functional Requirements

- Add and remove products from the personal wishlist.
- Find users and send, accept or reject friend requests.
- Share selected wishlist items with a friend.
- Open received shares and respond with reactions or comments.
- Show the current friend and request lists.

8.6.2 Non Functional Requirements

- Restrict wishlist and share access to the relevant user.
- Prevent duplicate wishlist items and invalid self-friendship records.
- Show a clear status for pending, accepted or rejected requests.

8.6.3 Tech Stack

React wishlist and friends pages, Express routes and controllers, SQLite.

8.6.4 Low Level Architecture

- A user saves a product through the wishlist route.
- The friends route stores a request and later updates its status.
- An accepted connection can receive a selected product share.
- Feedback is stored against the share and returned to its participants.

8.6.5 API Design

Method	Endpoint	Purpose
GET	/api/wishlist	Read wishlist
POST	/api/wishlist/items	Save product
DELETE	/api/wishlist/items/:productId	Remove product
POST	/api/friends/request	Send request
POST	/api/friends/:id/accept	Accept request
POST	/api/wishlist/shares	Create share

Method	Endpoint	Purpose
POST	/api/wishlist/shares/:shareId/feedback	Add feedback

8.6.6 DB Design

Table	Key Fields	Use
wishlists; wishlist_items	user_id; wishlist_id; product_id	Saved products
friendships	requester_id; receiver_id; status	Relationship state
wishlist_shares; wishlist_share_items	sender_id; receiver_id; share_id; product_id	Private share
wishlist_feedback	share_id; user_id; reaction; comment	Feedback

8.6.7 Screenshots

Fig 8.6.1 Personal Wishlist

Displays saved favourite products.

Allows removal from the list.

Keeps the list separate from cart.

Starts a product sharing action.

Fig 8.6.2 Friend Search and Requests

Lets a user find another account.

Creates a pending friend request.

Shows incoming and sent requests.

Supports acceptance or rejection.

Fig 8.6.3 Friends List

Shows accepted connections.
Provides an entry point for sharing.
Keeps pending requests distinct.
Represents the user social circle.

Fig 8.6.4 Shared Wishlist

Shows products sent by a friend.
Limits access to the intended recipient.
Preserves the sender and item context.
Allows feedback on the shared selection.

Fig 8.6.5 Share Feedback

Displays reactions and comments.
Lets a participant respond to the share.
Stores the response with the share record.
Supports collaborative style decisions.

8.7 Recommendations and Chatbot

8.7.0 Purpose and Scope

This module helps shoppers discover complementary clothing and obtain quick fashion guidance. Outfit suggestions use predefined matching rules for eligible products; the chatbot uses deterministic intent and catalogue logic.

8.7.1 Functional Requirements

- Return a complementary outfit for an eligible base product.
- Explain when an item is not eligible for outfit recommendations.
- Answer supported product, stock, styling and shopping questions.

- Offer links or guidance toward relevant catalogue and try-on pages.

8.7.2 Non Functional Requirements

- Do not present rule based results as learned predictions.
- Use current catalogue values when answering product questions.
- Return a controlled response for unsupported input or invalid product IDs.

8.7.3 Tech Stack

Express recommendation engine and chatbot controller, React product and chatbot components, SQLite.

8.7.4 Low Level Architecture

- The product page requests recommendations using the current product ID.
- The recommendation engine checks eligibility and selects complementary product slots.
- The chatbot receives a text message and matches supported intents.
- The API returns suggestions or a bounded fallback response for display.

8.7.5 API Design

Method	Endpoint	Purpose
GET	/api/recommendations/product/:productId	Get outfit suggestions
POST	/api/chatbot/message	Get assistant response

8.7.6 DB Design

Table	Key Fields	Use
products	id; category; color; price; stock	Current catalogue source
outfit_recommendations	base_product_id; recommended_product_id; slot_type	Stored recommendation links

8.7.7 Screenshots

Fig 8.7.1 Complementary Product Suggestions

Shows products that complete a look.
Starts from an eligible base item.
Displays matching garments by slot.
Supports navigation to another product.

Fig 8.7.2 Complete Look Selection

Presents the coordinated set of items.
Shows how the items combine.
Supports an outfit purchase action.
Uses the rule based selection result.

Fig 8.7.3 Style Assistant Chat

Opens the assistant from the interface.
Accepts a shopping or styling question.
Returns a defined answer from application logic.
May guide the user to relevant features.

8.8 Virtual Try-On

8.8.0 Purpose and Scope

Virtual try-on generates a preview of a supported garment on a user photograph. The Express API validates the request and contacts a separate local Python service that runs FASHN VTON 1.5.

8.8.1 Functional Requirements

- Report whether the local try-on service is ready.
- Accept a supported photo and selected product.
- Check garment eligibility and local garment image availability.

- Send the person and garment images to the Python service.
- Save and retrieve successful results for the authenticated user.

8.8.2 Non Functional Requirements

- Reject unsupported image types and missing uploads.
- Limit upload size and protect result access by owner.
- Report service unavailability without blocking ordinary shopping.

8.8.3 Tech Stack

React virtual try-on page, Express and Multer, local Python Flask FASHN VTON 1.5, SQLite.

8.8.4 Low Level Architecture

- The page checks VTON status and accepts an image upload.
- Express validates the product, garment asset and person image.
- The Python service generates an output image when ready.
- Express saves the output path and result record, then returns it to the page.

8.8.5 API Design

Method	Endpoint	Purpose
GET	/api/vton/status	Check service readiness
POST	/api/vton/try-on	Generate try-on result
GET	/api/vton/history	List own results
GET	/api/vton/results/:id	Get one owned result

8.8.6 DB Design

Table	Key Fields	Use
products	id; garment_image; vton_supported; vton_garment_category	Eligible garment metadata
try_on_results	user_id; product_id; generated_image_path; status	Generated result record

8.8.7 Screenshots

Fig 8.8.1 Virtual Try-On Entry

Shows service readiness and instructions.

Lets the shopper select a supported product.

Accepts a person photograph.

Starts the local image generation request.

Fig 8.8.2 Photo and Garment Selection

Shows the chosen person image.

Uses the product garment image on disk.

Checks file type and required inputs.

Submits both images for inference.

Fig 8.8.3 Generated Try-On Result

Displays a successful generated image.

Associates the output with its product.

Makes the preview available to the same user.

Shows an error if generation fails.

Fig 8.8.4 Try-On History

Lists recent generated results.
Uses the signed-in user identifier.
Allows opening a saved result.
Does not expose another user result.

8.9 Admin Dashboard

8.9.0 Purpose and Scope

The administrator module provides an account restricted view of catalogue and application activity. It offers summary metrics, users, purchases, virtual try-on activity and limited product updates.

8.9.1 Functional Requirements

- Require an ADMIN role before exposing routes.
- Display metrics and recent activity.
- List products, users and purchases for administration.
- Update supported product fields through the admin route.
- Show virtual try-on and email status information.

8.9.2 Non Functional Requirements

- Enforce the role on the backend even if a page route is hidden.
- Avoid exposing password hashes in user summaries.
- Validate editable product fields before writing them.

8.9.3 Tech Stack

React admin pages, Express requireAdmin middleware, admin controller and SQLite.

8.9.4 Low Level Architecture

- The admin page sends a JWT with dashboard requests.
- The requireAdmin middleware verifies the account role.
- Controller queries summarize current database records.
- Product changes are validated and written through the admin endpoint.

8.9.5 API Design

Method	Endpoint	Purpose
GET	/api/admin/metrics	Read dashboard metrics
GET	/api/admin/products	List products
PATCH	/api/admin/products/:id	Update product
GET	/api/admin/users	List users
GET	/api/admin/purchases	List purchases
GET	/api/admin/vton-stats	Read try-on metrics

8.9.6 DB Design

Table	Key Fields	Use
users	id; role; email	Role and user summary
products	id; price; stock; status	Catalogue editing
purchases	id; user_id; total_amount	Purchase summary
try_on_results	user_id; product_id; status	Try-on activity

8.9.7 Screenshots

Fig 8.9.1 Admin Dashboard Overview

Displays restricted application summaries.

Shows current metrics from SQLite.

Provides navigation to management sections.

Requires an ADMIN account.

Fig 8.9.2 Admin Product Management

Lists catalogue products for review.

Shows editable product fields.

Submits supported changes to the API.

Reflects updated database values.

Fig 8.9.3 Admin Users and Purchases

Shows account and purchase summaries.

Supports operational visibility.

Keeps password hashes out of display.

Limits the view to administrators.

Fig 8.9.4 Admin Activity and VTON Metrics

Shows recent activity indicators.

Summarizes virtual try-on records.

Helps inspect service use and status.

Uses protected dashboard endpoints.

8.10 Theme and Language Preferences

8.10.0 Purpose and Scope

This module lets a user select the application appearance, interface language and communication preferences. English, Tamil and Hindi translations are provided by the current frontend language context.

8.10.1 Functional Requirements

- Switch between available interface languages.
- Remember the selected language across page reloads.
- Change among supported appearance themes.
- Save email, SMS and WhatsApp notification preferences.
- Retrieve the signed-in user settings and delivery history.

8.10.2 Non Functional Requirements

- Use a supported language when a saved value is unavailable.

- Keep account preferences associated with the correct user.
- Avoid claiming external delivery from a preference switch alone.

8.10.3 Tech Stack

React LanguageContext, ThemeContext and SettingsDrawer, Express settings routes, SQLite.

8.10.4 Low Level Architecture

- The user opens settings and changes a language or appearance choice.
- The frontend updates its context and local persistence for immediate display.
- Account communication preferences are sent to the protected settings API.
- The database stores the supported user preference fields.

8.10.5 API Design

Method	Endpoint	Purpose
GET	/api/settings	Read user settings
PUT	/api/settings	Save preferences
GET	/api/settings/notifications/history	Read delivery history

8.10.6 DB Design

Table	Key Fields	Use
users	id; theme_preference; language_preference; email_notifications; sms_notifications; whatsapp_notifications	Saved user choices
notification_deliveries	user_id; channel; status	Communication history

8.10.7 Screenshots

Fig 8.10.1 Settings Drawer

Groups appearance and communication choices.

Shows current saved values.

Allows changes from a single panel.

Provides feedback after a save.

Fig 8.10.2 Language Selector

Offers English, Tamil and Hindi.

Changes visible interface labels.

Remembers the choice across reloads.

Falls back when a translation key is absent.

Fig 8.10.3 Theme Selector

Offers the available appearance themes.

Applies the chosen style to the interface.

Keeps the selected preference.

Does not alter shopping records.

Fig 8.10.4 Communication Preferences

Shows channel switches for notifications.

Stores each enabled or disabled choice.

Controls attempted optional delivery.

Delivery success depends on provider configuration.

CONCLUSION

StyleHub brings the main operations of a men fashion shopping application into one system: protected accounts, product browsing, cart and purchase handling, wishlists, friend sharing, rule based suggestions, administration, and saved preferences. The SQLite schema and REST routes provide a coherent basis for these modules.

Virtual try-on extends the project through a local third party model, while Google sign-in and communication channels depend on external configuration. The report diagrams and screenshot spaces support later submission updates without changing the underlying technical description.